

RAG : Retrieval-Augmented Generation

Troisième chapitre du module IA Générative

Enseignant : Haythem Ghazouani

Module : IA Générative

14 avril 2026

Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- définir précisément ce qu'est le **RAG** ;
- expliquer les notions de document, chunk, embedding, index, retriever et contexte ;
- comparer plusieurs choix technologiques comme **FAISS** et **ChromaDB** ;
- comprendre le rôle de **LangChain**, **PromptTemplate** et **LangGraph** ;
- lire et expliquer des exemples Python de pipeline RAG ;
- concevoir un chatbot RAG simple avec l'API **Groq**.

Plan

- 1 Pourquoi le RAG ?
- 2 Les notions fondamentales
- 3 Architecture d'un système RAG
- 4 Technologies et choix possibles
- 5 RAG et LangChain
- 6 Exemples Python progressifs
- 7 Bonnes pratiques RAG
- 8 LangGraph
- 9 Exemple complet de chatbot RAG
- 10 Synthèse et révision

Pourquoi les LLM seuls ne suffisent pas ?

Un grand modèle de langage peut :

- répondre de manière fluide ;
- résumer, traduire, expliquer ;
- générer du code ou du texte.

Mais il a aussi des limites :

- il ne connaît pas forcément vos documents internes ;
- il peut être dépassé sur des informations récentes ;
- il peut halluciner ;
- il n'a pas naturellement accès à une base documentaire spécifique.

Exemple concret du problème

Imaginons un chatbot d'université.

Un étudiant demande :

Quelle est la procédure exacte d'inscription administrative dans mon établissement ?

Sans RAG : le modèle peut répondre de manière générique, voire fausse.

Avec RAG : le système va chercher les documents de l'université, retrouver les bons passages, puis répondre à partir de ces sources.

Définition du RAG

RAG = *Retrieval-Augmented Generation*

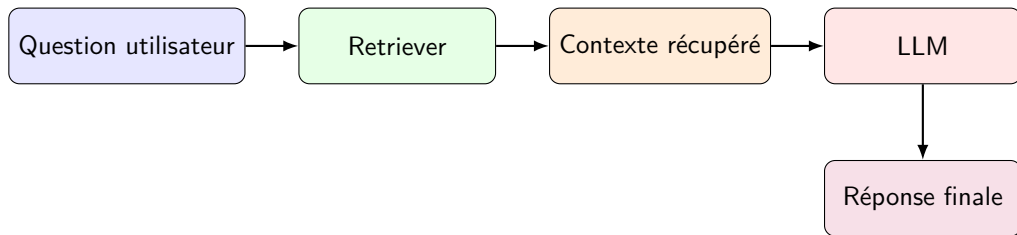
En français :

génération augmentée par récupération d'information.

Idée centrale :

- ① récupérer des documents pertinents ;
- ② injecter ce contenu dans le contexte du modèle ;
- ③ générer une réponse fondée sur ces informations.

Figure : principe général du RAG



Lecture : le LLM n'invente pas tout seul ; il s'appuie sur les informations retrouvées.

Les bénéfices du RAG

Le RAG permet de :

- connecter un LLM à vos documents ;
- améliorer la pertinence des réponses ;
- réduire certaines hallucinations ;
- mettre à jour la base documentaire sans réentraîner le modèle ;
- citer ou retrouver les sources utilisées.

Document, chunk, requête

Trois notions de base :

- **document** : un texte source, un PDF, une page web, un rapport ;
- **chunk** : un morceau plus petit d'un document ;
- **requête** : la question ou le besoin de l'utilisateur.

Pourquoi découper en *chunks* ?

- un document complet peut être trop long ;
- la recherche est souvent plus précise sur des morceaux courts ;
- le contexte injecté dans le prompt devient plus contrôlable.

Chunking : pourquoi et comment ?

Chunking = découpage des documents en unités exploitables.

Quelques stratégies :

- découpage fixe par nombre de caractères ;
- découpage par paragraphes ;
- découpage sémantique ;
- découpage avec *overlap* pour conserver le contexte.

Overlap : on fait se chevaucher deux chunks pour éviter de casser une idée importante entre deux morceaux.

Exemple simple de chunking

Document :

“Le module d’IA générative commence le 10 septembre. Les étudiants doivent déposer leur dossier avant le 5 septembre. Les frais d’inscription sont payables en ligne.”

Chunk 1 :

Le module d’IA générative commence le 10 septembre. Les étudiants doivent déposer leur dossier avant le 5 septembre.

Chunk 2 :

Les étudiants doivent déposer leur dossier avant le 5 septembre. Les frais d’inscription sont payables en ligne.

Qu'est-ce qu'un embedding ?

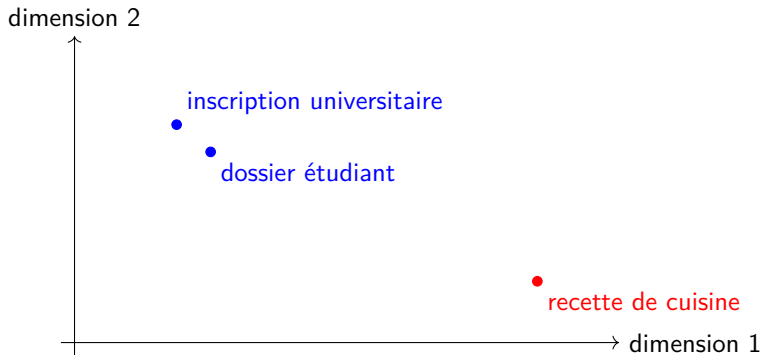
Un **embedding** est une représentation numérique d'un texte sous la forme d'un vecteur.

Intuition :

- deux textes proches en sens ont souvent des vecteurs proches ;
- deux textes très différents ont des vecteurs éloignés.

Les embeddings servent donc à faire de la **recherche sémantique**, c'est-à-dire rechercher par sens plutôt que seulement par mots exacts.

Figure : idée intuitive des embeddings



Lecture : les textes proches en sens se retrouvent proches dans l'espace vectoriel.

Pourquoi les embeddings sont-ils essentiels en RAG ?

Sans embeddings, la recherche repose surtout sur les mots présents exactement.

Avec embeddings, une question comme :

Comment m'inscrire ?

peut retrouver un passage contenant :

Procédure d'inscription administrative et dépôt du dossier.

Idée clé : la recherche sémantique capte le sens, pas seulement les mots identiques.

Similarité, distance et recherche vectorielle

Pour retrouver des chunks pertinents, on compare les vecteurs.

Mesures fréquentes :

- **distance euclidienne (L2)** ;
- **cosine similarity** ;
- **inner product** ou produit scalaire.

En pratique :

- plus deux vecteurs sont proches, plus les textes sont jugés similaires ;
- le système retourne les **top-k** chunks les plus proches.

Index vectoriel : structure qui stocke et organise les embeddings pour permettre une recherche rapide.

Retriever : composant qui interroge l'index pour récupérer les documents pertinents.

Reranker : composant optionnel qui reclasse les documents récupérés pour améliorer encore la pertinence.

Le pipeline typique est :

question → embedding de la question → recherche → top-k chunks

En RAG, le mot **contexte** peut désigner plusieurs choses :

- le contenu récupéré depuis la base documentaire ;
- l'historique de conversation ;
- les consignes système ou les règles métier ;
- la fenêtre maximale de contexte du modèle.

Très important : le RAG injecte dans le prompt un **contexte externe** choisi dynamiquement.

Les deux grandes phases du RAG

Un système RAG a généralement deux phases :

- 1 **Indexation** : on prépare les documents.
- 2 **Question-réponse** : on répond à l'utilisateur.

Phase 1 : indexation

Pendant l'indexation, on :

- ➊ charge les documents ;
- ➋ les nettoie ;
- ➌ les découpe en chunks ;
- ➍ calcule les embeddings ;
- ➎ stocke les vecteurs dans un index ou une base vectorielle.

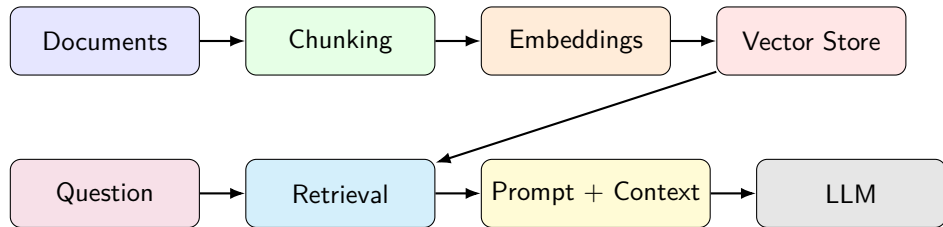
Cette phase peut être faite une fois, puis mise à jour quand les documents changent.

Phase 2 : question-réponse

Quand l'utilisateur pose une question :

- ① on calcule l'embedding de la question ;
- ② on cherche les chunks les plus proches ;
- ③ on construit un prompt avec la question et le contexte ;
- ④ le LLM génère la réponse.

Figure : architecture complète d'un pipeline RAG



Pour construire un RAG, on choisit en général :

- un modèle d'embedding ;
- un vector store ou un index vectoriel ;
- un framework d'orchestration ;
- un LLM de génération ;
- éventuellement un outil de workflow ou d'agents.

FAISS : qu'est-ce que c'est ?

FAISS = *Facebook AI Similarity Search*

C'est une bibliothèque optimisée pour la recherche de similarité sur des vecteurs denses.

Points forts :

- très rapide ;
- très utilisée en recherche et en prototypes ;
- permet différents types d'index ;
- supporte des volumes importants et des optimisations avancées.

FAISS : quand l'utiliser ?

FAISS est particulièrement adapté si :

- vous voulez un contrôle fin sur l'indexation ;
- vous travaillez localement en Python ;
- vous avez besoin d'un moteur très performant de similarité ;
- vous gérez vous-même la logique autour du stockage et des métadonnées.

Limite : FAISS n'est pas une base documentaire complète avec expérience produit prête à l'emploi ; c'est surtout un moteur de recherche vectorielle.

ChromaDB : qu'est-ce que c'est ?

ChromaDB est une base vectorielle pensée pour les applications d'IA.

Elle permet de :

- créer des collections ;
- stocker documents, embeddings et métadonnées ;
- faire des requêtes par texte ou par embedding ;
- persister les données plus simplement qu'un bricolage maison.

ChromaDB est souvent appréciée pour sa simplicité dans les prototypes et petits à moyens projets.

FAISS ou ChromaDB ?

Critère	FAISS	ChromaDB
Nature	bibliothèque d'index vectoriel	base/vector store orienté IA
Simplicité	plus bas niveau	plus simple à manipuler
Métadonnées	à gérer souvent soi-même	support intégré
Persistence	possible mais plus technique	simple à mettre en place
Cas d'usage	performance et contrôle fin	prototypage rapide et RAG applicatif

Autres options possibles

Au-delà de FAISS et ChromaDB, on peut aussi utiliser :

- **Qdrant** ;
- **Weaviate** ;
- **Pinecone** ;
- **Milvus** ;
- **Elastic** avec recherche vectorielle ;
- des approches hybrides texte + vecteurs.

Le bon choix dépend du volume, du budget, de la persistance, du cloud et des besoins de filtrage.

Le choix du modèle d'embedding

Le modèle d'embedding est déterminant.

On peut utiliser :

- des modèles locaux comme ceux de `sentence-transformers` ;
- des embeddings via API ;
- des modèles généralistes ou spécialisés.

Critères de choix :

- qualité sémantique ;
- langue supportée ;
- coût ;
- vitesse ;
- taille du vecteur.

Très bien comprendre les embeddings

Pour bien comprendre un RAG, il faut bien comprendre les embeddings.

Un embedding :

- n'est pas une phrase ;
- n'est pas un résumé ;
- est une **signature numérique du sens**.

Deux phrases sémantiquement proches produisent souvent des vecteurs voisins, même si elles ne partagent pas exactement les mêmes mots.

Exemple concret d'embeddings

Les phrases suivantes peuvent être proches :

- “Comment payer mes frais d’inscription ?”
- “Quelle est la procédure de paiement universitaire ?”

Elles peuvent être éloignées de :

- “Comment installer Python sous Windows ?”

Pourquoi ? parce que le modèle d'embedding capture le thème et l'intention.

Les limites des embeddings

Les embeddings sont puissants, mais pas magiques.

Difficultés possibles :

- ambiguïtés de langue ;
- documents trop longs ou trop mal chunkés ;
- embeddings peu adaptés au domaine ;
- perte de précision sur des détails très spécifiques ;
- bruit si le corpus est mal nettoyé.

Pourquoi utiliser LangChain ?

LangChain fournit des briques réutilisables pour construire des applications LLM.

Dans un RAG, LangChain aide à :

- gérer les documents ;
- brancher des embeddings ;
- utiliser des vector stores ;
- construire des retrievers ;
- écrire des prompts dynamiques ;
- enchaîner les étapes du pipeline.

Quelques notions essentielles :

- **Document** : texte + métadonnées ;
- **Embeddings** : calcul de vecteurs ;
- **VectorStore** : stockage et recherche ;
- **Retriever** : interface de récupération ;
- **PromptTemplate** : prompt paramétrable ;
- **Runnable** : composant chaînable.

Qu'est-ce qu'un PromptTemplate ?

Un **PromptTemplate** est un prompt avec des variables.

Exemple :

Réponds à la question suivante à partir du contexte.

Contexte : {context}

Question : {question}

Intérêt : on réutilise le même squelette de prompt en injectant dynamiquement la question et le contexte.

Rôle du contexte dans le prompt final

Le prompt final d'un RAG contient souvent :

- une consigne système ;
- le contexte récupéré ;
- la question de l'utilisateur ;
- parfois des règles métier ;
- parfois une consigne de citation de sources.

Exemple de consigne :

Réponds uniquement à partir du contexte fourni. Si l'information n'y figure pas, dis-le explicitement.

Exemple 1 : embeddings en Python

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

texts = [
    "Procédure d'inscription universitaire",
    "Paieement des frais de scolarite",
    "Installation de Python sur Windows",
]

vectors = model.encode(texts)
print(vectors.shape)
```

Dans cet exemple :

- on charge un modèle d'embedding ;
- on transforme plusieurs textes en vecteurs ;
- chaque ligne de vectors représente un texte.

Ces vecteurs pourront ensuite être stockés dans un index FAISS ou une base comme ChromaDB.

Exemple 2 : recherche simple avec FAISS

```
import faiss
from sentence_transformers import SentenceTransformer

docs = [
    "Les frais d'inscription sont payables en ligne.",
    "Le dossier doit etre depose avant le 5 septembre.",
]

model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(docs).astype("float32")

index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)

query = "Comment payer mon inscription ?"
query_vec = model.encode([query]).astype("float32")

distances, indices = index.search(query_vec, k=2)
print(indices)
print([docs[i] for i in indices[0]])
```

Ce code montre les étapes minimales :

- ① calculer les embeddings des documents ;
- ② créer un index ;
- ③ ajouter les vecteurs ;
- ④ transformer la question en embedding ;
- ⑤ rechercher les chunks les plus proches.

Exemple 3 : ChromaDB en natif

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(name="reglement_universite")

collection.add(
    ids=["doc1", "doc2"],
    documents=[
        "Le dossier doit etre depose avant le 5 septembre.",
        "Les frais sont reglables sur la plateforme en ligne."
    ],
    metadatas=[{"source": "guide.pdf"}, {"source": "faq.pdf"}]
)

results = collection.query(query_texts=["Comment payer les frais ?"],
                           n_results=2)

print(results["documents"])
print(results["metadatas"])
```


Pourquoi ChromaDB est pratique ?

ChromaDB est pratique car il permet facilement de :

- stocker texte + métadonnées ;
- persister localement ;
- faire des requêtes par texte ;
- réutiliser les collections ;
- prototyper rapidement un RAG.

Exemple 4 : PromptTemplate avec LangChain

```
from langchain_core.prompts import PromptTemplate

template = """
Tu es un assistant universitaire.
Reponds uniquement a partir du contexte.

Contexte :
{context}

Question :
{question}
"""

prompt = PromptTemplate.from_template(template)

final_prompt = prompt.format(
    context="Les frais d'inscription sont payables en ligne.",
    question="Comment payer mes frais ?"
)

print(final_prompt)
```

Pourquoi PromptTemplate est important en RAG ?

Il permet de :

- standardiser les réponses ;
- injecter dynamiquement le contexte récupéré ;
- mieux contrôler le comportement du modèle ;
- imposer des règles comme “si absent du contexte, dis-le”.

Exemple 5 : LangChain + Chroma + Retriever

```
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_core.documents import Document

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

documents = [
    Document(page_content="Le dossier doit etre depose avant le 5 septembre."),
    Document(page_content="Les frais d'inscription sont payables en ligne."),
]

vectorstore = Chroma.from_documents(
    documents=documents,
    embedding=embeddings,
    persist_directory="./chroma_langchain"
)

retriever = vectorstore.as_retriever(search_kwargs={"k": 2})
docs = retriever.invoke("Comment payer les frais ?")
print(docs)
```

Le **retriever** est l'interface qui transforme votre vector store en composant de recherche.

Dans LangChain, on l'utilise pour :

- récupérer les documents les plus pertinents ;
- choisir le nombre de résultats k ;
- brancher facilement le résultat dans un pipeline de génération.

Exemple 6 : appel direct a l'API Groq

```
import os
from groq import Groq

client = Groq(api_key=os.environ["GROQ_API_KEY"])

chat_completion = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "system", "content": "Tu es un assistant pedagogique."},
        {"role": "user", "content": "Explique le RAG en 5 points simples."}
    ]
)

print(chat_completion.choices[0].message.content)
```

Pourquoi Groq ici ?

L'API **Groq** est intéressante dans un chapitre RAG car :

- elle permet d'utiliser rapidement des modèles de chat ;
- elle est adaptée à des démonstrations de chatbot ;
- elle peut être utilisée directement via SDK Python ;
- elle s'intègre aussi avec LangChain.

Exemple 7 : LangChain + ChatGroq

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    temperature=0
)

response = llm.invoke("Explique le RAG simplement.")
print(response.content)
```


Exemple 8 : mini pipeline RAG avec LangChain

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

prompt = PromptTemplate.from_template("""
Reponds a partir du contexte.
Contexte: {context}
Question: {question}
""")

rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)
```

Lecture du mini pipeline RAG

Ici :

- le `retriever` récupère les documents ;
- `format_docs` les transforme en texte ;
- `PromptTemplate` construit le prompt final ;
- le LLM génère la réponse ;
- `StrOutputParser` renvoie une chaîne de texte simple.

Les paramètres qui influencent beaucoup la qualité

Plusieurs choix changent fortement les résultats :

- taille des chunks ;
- overlap ;
- qualité du modèle d'embedding ;
- valeur de k ;
- formulation du prompt ;
- qualité et propreté des documents.

Erreurs fréquentes en RAG

- chunks trop grands ;
- chunks trop petits ;
- documents non nettoyés ;
- embeddings mal adaptés à la langue ou au domaine ;
- absence de consigne claire dans le prompt ;
- croire que le retrieval est correct sans l'évaluer.

Comment améliorer un RAG ?

Pistes d'amélioration :

- améliorer le chunking ;
- ajouter des métadonnées ;
- filtrer par source ou type de document ;
- utiliser un reranker ;
- faire du *hybrid search* lexical + vectoriel ;
- demander au modèle de citer ses sources.

Pourquoi LangGraph ?

LangGraph sert à construire des workflows et agents **stateful**, c'est-à-dire avec un état partagé entre plusieurs étapes.

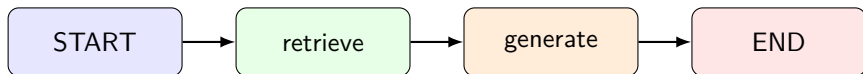
Il est utile quand un simple pipeline linéaire ne suffit plus :

- plusieurs noeuds doivent coopérer ;
- on veut des branches conditionnelles ;
- on veut des boucles ou une logique d'orchestration ;
- on veut des workflows plus robustes.

Les notions importantes sont :

- **State** : l'état partagé ;
- **Node** : une étape de traitement ;
- **Edge** : un lien entre deux étapes ;
- **START** et **END** : début et fin du graphe ;
- logique conditionnelle ou séquentielle.

Figure : pipeline RAG vu comme graphe



Exemple 9 : LangGraph minimal pour un RAG

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class RAGState(TypedDict):
    question: str
    context: str
    answer: str

def retrieve_node(state: RAGState):
    docs = retriever.invoke(state["question"])
    return {"context": "\n\n".join(doc.page_content for doc in docs)}

def generate_node(state: RAGState):
    prompt_text = prompt.format(context=state["context"],
                                question=state["question"])
    answer = llm.invoke(prompt_text).content
    return {"answer": answer}

graph = StateGraph(RAGState)
graph.add_node("retrieve", retrieve_node)
graph.add_node("generate", generate_node)
graph.add_edge(START, "retrieve")
```

Pourquoi ce passage a LangGraph est important ?

Ce code montre comment :

- séparer clairement la récupération et la génération ;
- partager un état commun ;
- rendre le pipeline plus lisible ;
- préparer des extensions plus complexes.

On pourra ensuite ajouter :

- un noeud de vérification ;
- un noeud de reformulation ;
- une branche si aucun document n'est trouvé.

Exemple d'évolution avec LangGraph

Un graphe plus avancé peut :

- ① reformuler la question ;
- ② récupérer les documents ;
- ③ vérifier leur pertinence ;
- ④ générer une réponse ;
- ⑤ demander une nouvelle recherche si le contexte est insuffisant.

C'est là que LangGraph devient très utile pour des assistants RAG robustes.

On veut construire un chatbot qui répond à des questions sur un règlement universitaire.

Composants choisis :

- documents PDF ou textes institutionnels ;
- embeddings locaux ;
- ChromaDB pour le stockage ;
- LangChain pour l'orchestration ;
- Groq pour le modèle de génération ;
- éventuellement LangGraph pour le workflow.

Exemple 10 : squelette de chatbot RAG

```
question = "Quelle est la date limite de depot du dossier ?"

docs = retriever.invoke(question)
context = "\n\n".join(doc.page_content for doc in docs)

prompt_text = prompt.format(
    context=context,
    question=question
)

answer = llm.invoke(prompt_text).content
print(answer)
```

Que faut-il surveiller dans ce chatbot ?

- la qualité du corpus ;
- la qualité de la récupération ;
- la transparence sur les sources ;
- les cas où aucune réponse fiable n'existe ;
- la sécurité des données ;
- le coût et la latence.

Liens pour tester et documenter

- Groq API docs : <https://console.groq.com/docs/api-reference>
- Groq Text Chat : <https://console.groq.com/docs/text-chat>
- Groq Playground : <https://console.groq.com/playground>
- Groq Python SDK : <https://github.com/groq/groq-python>
- LangChain docs : <https://python.langchain.com/docs/tutorials/retrievers/>
- PromptTemplate : <https://reference.langchain.com/python/langchain-core/prompts/prompt/PromptTemplate>
- ChatGroq : <https://docs.langchain.com/oss/python/integrations/chat/groq>
- LangGraph : <https://docs.langchain.com/oss/python/langgraph/overview>
- FAISS : <https://faiss.ai/index.html>
- ChromaDB : <https://docs.trychroma.com/docs/embeddings/embedding-functions>

Carte mentale du chapitre

- RAG = retrieval + génération ;
- chunking = découpage exploitable des documents ;
- embedding = représentation vectorielle du sens ;
- vector store = stockage et recherche de vecteurs ;
- retriever = récupération des chunks pertinents ;
- PromptTemplate = construction propre du prompt ;
- LangChain = briques de pipeline ;
- LangGraph = orchestration d'un workflow ou agent plus complexe.

10 notions à retenir

- 1 RAG
- 2 chunk
- 3 overlap
- 4 embedding
- 5 similarité vectorielle
- 6 vector store
- 7 retriever
- 8 PromptTemplate
- 9 context
- 10 LangGraph

- ❶ Pourquoi un LLM seul ne suffit-il pas toujours ?
- ❷ Quel est le rôle des embeddings ?
- ❸ Quelle différence entre FAISS et ChromaDB ?
- ❹ À quoi sert un retriever ?
- ❺ Pourquoi le chunking influence-t-il fortement la qualité du RAG ?
- ❻ Quel est le rôle de PromptTemplate ?
- ❼ Pourquoi utiliser LangGraph au-delà d'un simple pipeline linéaire ?

Objectif : comparer deux configurations RAG.

Travail demandé :

- groupe 1 : indexation simple avec FAISS ;
- groupe 2 : indexation simple avec ChromaDB ;
- comparer simplicité, lisibilité du code et type de résultats ;
- proposer des améliorations sur chunking, prompt ou retrieval.

- Le RAG est l'une des techniques les plus importantes pour connecter un LLM à des connaissances externes.
- Sa qualité dépend autant des documents et des embeddings que du modèle génératif.
- FAISS, ChromaDB, LangChain et LangGraph répondent à des besoins complémentaires.
- Un bon système RAG est à la fois technique, méthodique et évalué avec rigueur.

Merci

Chapitre 3 : RAG